SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH. Durga Prasad	**Reg. No.:**	192425305
Section / Batch:	2024	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Q8. Activity Selection – Scheduling Optimization

A software company must schedule multiple client meetings in limited time slots, ensuring maximum number of non-overlapping meetings. Analyze how the Activity Selection problem can be solved using a Greedy approach. Design a solution by explaining the selection strategy based on finishing times. Justify why the greedy method provides an optimal solution. Suggest suitable tools or programming techniques for implementation. Analyze time complexity and interpret the results in terms of efficient scheduling.

Code of Conduct

I CH Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH. Durga Prasad

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				

Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q8. Activity Selection – Scheduling Optimization

1. Introduction

The **Activity Selection Problem** is a well-known optimization problem in **Design and Analysis of Algorithms (DAA)**. It is used when a set of activities or tasks are given, where each activity has a **starting time** and a **finishing time**. The objective is to select the **maximum possible number of activities without overlapping** with each other.

In this problem, a software company needs to schedule several **client meetings** within limited working hours. Since two meetings cannot be conducted at the same time, the company must choose meetings carefully so that the maximum number of meetings can be completed. The **Greedy Algorithm** is an efficient technique for solving this problem. It selects the activity that **finishes earliest**, because this leaves the largest possible amount of time for the remaining activities.

2. Problem Description

Suppose a software company has several client meetings planned during a working day. Each meeting requires a particular time interval.

For example:

Meeting	Start Time	Finish Time
M1	9:00 AM	10:00 AM
M2	9:30 AM	11:00 AM
M3	10:00 AM	11:30 AM
M4	11:00 AM	12:00 PM
M5	11:30 AM	1:00 PM
M6	12:00 PM	1:30 PM
M7	1:00 PM	2:00 PM

The company cannot select meetings that overlap. Therefore, the goal is to find a sequence of meetings that gives the **maximum number of non-overlapping meetings**.

3. Objective

The main objectives of the Activity Selection problem are:

Objective	Description
Maximize meetings	Select the largest possible number of client meetings.
Avoid conflicts	Ensure that selected meetings do not overlap.
Efficient scheduling	Make maximum use of the available working hours.
Reduce idle time	Avoid unnecessary gaps where possible.
Fast computation	Obtain the solution efficiently even when many meetings are given.

4. Greedy Strategy

The Activity Selection problem can be solved using a **Greedy Approach**.

The basic strategy is:

Always select the activity that has the earliest finishing time among the remaining compatible activities.

The reason for selecting the earliest finishing meeting is that it leaves more time available for the remaining meetings.

The steps are:

1. List all meetings with their start and finish times.
2. Sort the meetings according to their **finish time** in ascending order.
3. Select the first meeting because it finishes earliest.
4. Check the remaining meetings one by one.
5. Select a meeting if its start time is **greater than or equal to** the finish time of the previously selected meeting.
6. Continue until all meetings have been checked.
7. The selected meetings form the maximum set of non-overlapping activities.

5. Example of Greedy Selection

Consider the following meetings:

Meeting	Start	Finish	Selected?	Reason
M1	9:00	10:00	**Yes**	Finishes earliest
M2	9:30	11:00	No	Starts before M1 finishes
M3	10:00	11:30	**Yes**	Starts when M1 finishes
M4	11:00	12:00	No	Overlaps with M3
M5	11:30	1:00	No	Overlaps with M3

M6	12:00	1:30	**Yes**	Starts after M3 finishes
M7	1:00	2:00	No	Overlaps with M6

Therefore, the selected meetings are:

M1 → M3 → M6

The company can successfully conduct **3 non-overlapping meetings**.

6. Selection Process in Detail

After sorting according to finishing time:

Step	Meeting	Start	Finish	Previous Finish	Decision
1	M1	9:00	10:00	—	Select
2	M2	9:30	11:00	10:00	Reject
3	M3	10:00	11:30	10:00	Select
4	M4	11:00	12:00	11:30	Reject
5	M5	11:30	1:00	11:30	Select
6	M6	12:00	1:30	1:00	Reject
7	M7	1:00	2:00	1:00	Select

Depending on whether the boundary condition allows a meeting starting exactly when another ends, the compatibility rule is:

Start Time ≥ Previous Finish Time

Using that rule, the correct sequence from the given sorted intervals is:

M1 → M3 → M5 → M7

Thus, **4 meetings** can be scheduled.

7. Greedy Algorithm

Pseudocode

ActivitySelection(start[], finish[], n)

1. Sort all activities according to finish time.
2. Select the first activity.
3. Set lastFinish = finish of selected activity.
4. For i = 2 to n:

    If start[i] >= lastFinish:

        Select activity i

        lastFinish = finish[i]

5. Return selected activities.

8. Algorithm Explanation

The algorithm first sorts all activities based on their **finishing time**. The activity with the smallest finishing time is selected first. After selecting an activity, the algorithm compares the starting time of every subsequent activity with the finishing time of the previously selected activity.

If:

Start of current activity ≥ Finish of previous activity

then the current activity is selected.

Otherwise, it is rejected because it overlaps with the already selected activity.

This process continues until all activities are examined.

9. Why Earliest Finishing Time is Used

There are different possible strategies for selecting meetings, such as:

Strategy	**Problem**
Select earliest starting meeting	May occupy a large portion of the available time.
Select shortest meeting	Short duration alone does not guarantee the maximum number of meetings.
Select meeting with fewest conflicts	Can produce a non-optimal schedule.
Select latest finishing meeting	Leaves less time for subsequent activities.
Select earliest finishing meeting	**Leaves maximum remaining time and gives an optimal solution.**

Therefore, **earliest finishing time** is the correct greedy criterion for the standard Activity Selection Problem.

10. Why the Greedy Method is Optimal

The greedy method provides an optimal solution because of the **Greedy Choice Property**.

Suppose we choose the activity that finishes earliest. Since it finishes before or at the same time as every other possible first activity, choosing it cannot reduce the amount of time available for the remaining activities.

After selecting the earliest-finishing activity, the same problem remains: select the maximum number of activities that start after its finishing time. The greedy strategy is applied repeatedly to this smaller problem.

This demonstrates the **optimal substructure** of the problem.

Greedy Choice Property

The first selected activity should be the one with the **earliest finish time**.

Optimal Substructure

After selecting the first activity, the remaining problem is another Activity Selection problem involving only activities that start after the selected activity finishes.

Therefore:

Optimal solution = Greedy choice + Optimal solution to the remaining activities

11. Complexity Analysis

Let **n** be the number of meetings.

Operation	Time Complexity
Reading activities	O(n)
Sorting by finish time	O(n log n)
Selecting compatible activities	O(n)
Total	**O(n log n)**

The sorting operation is the most time-consuming step.

Therefore, the overall time complexity is:

O(n log n)

If the meetings are already sorted by finishing time, the selection process itself takes only:

O(n)

12. Space Complexity

The algorithm requires space to store the meeting information.

Component	Space
Start times	O(n)
Finish times	O(n)
Selected activities	O(n)
Additional algorithm space	O(1) to O(n), depending on implementation
Overall	**O(n)**

---

13. Suitable Programming Tools

The Activity Selection problem can be implemented using several programming languages and tools.

Tool/Technology	Application
Python	Simple implementation using lists and sorting
C	Efficient implementation using arrays and structures
C++	Uses vectors, structures/classes and sorting functions
Java	Uses arrays, classes and collection sorting
VS Code	Coding and testing environment
Dev-C++	Suitable for C/C++ implementation
Jupyter Notebook	Useful for Python experimentation and demonstration

For a student-level implementation, **Python or C++** is recommended because the sorting and selection logic can be implemented easily.

14. Python Implementation

```
activities = [
    ("M1", 9, 10),
    ("M2", 9.5, 11),
    ("M3", 10, 11.5),
    ("M4", 11, 12),
    ("M5", 11.5, 13),
    ("M6", 12, 13.5),
    ("M7", 13, 14)
]
activities.sort(key=lambda x: x[2])
selected = []
last_finish = 0
for activity in activities:
    name, start, finish = activity
    if start >= last_finish:
        selected.append(activity)
        last_finish = finish
print("Selected Meetings:")

for activity in selected:
    print(activity)
print("Maximum number of meetings:", len(selected))
```

Expected Result

Selected Meeting	Start	Finish
M1	9:00	10:00
M3	10:00	11:30
M5	11:30	1:00
M7	1:00	2:00

Maximum number of meetings = 4

15. Advantages of the Greedy Approach

Advantage	Explanation
Simple	The algorithm is easy to understand and implement.
Fast	It has an efficient O(n log n) complexity.
Optimal	Produces the maximum number of non-overlapping activities.
Scalable	Can handle a large number of meetings efficiently.
Low computation	Does not require complex dynamic programming tables.
Practical	Suitable for real-world scheduling applications.

16. Limitations

The standard Activity Selection algorithm assumes that the primary objective is to **maximize the number of activities**.

It does not directly consider:

- Different priorities of meetings
- Different profits or revenues
- Meeting importance
- Employee availability
- Multiple conference rooms
- Travel time between locations
- Different resource requirements

For example, if one client meeting generates ₹1,00,000 revenue while another generates ₹5,000, simply maximizing the number of meetings may not be the best business decision. In such cases, problems such as **Weighted Activity Selection** may be more appropriate.

17. Real-World Application

The Activity Selection technique can be applied to many scheduling situations.

Application	Use
Client meetings	Schedule maximum number of meetings without conflicts
Interview scheduling	Arrange candidate interviews
Conference rooms	Assign meetings to available rooms
Online consultations	Schedule customer or patient consultations
Project tasks	Arrange independent tasks within time constraints
Examination scheduling	Select non-conflicting examination sessions
Job scheduling	Schedule compatible jobs
Event management	Organize events within available time
Resource allocation	Efficiently use limited resources

18. Interpretation of Results

The result of the greedy algorithm represents the **maximum number of non-overlapping meetings** that can be completed during the available time.

For the example, the company has **7 possible meetings**, but because several meetings overlap, it cannot conduct all of them. The greedy algorithm selects:

M1 → M3 → M5 → M7

Therefore, **4 meetings** can be successfully scheduled.

This means the company makes efficient use of its working hours while avoiding meeting conflicts. The selection of the earliest-finishing meeting at each step leaves maximum time available for subsequent meetings.

19. Overall Analysis

The Activity Selection problem demonstrates how a simple local decision can lead to a globally optimal solution. By selecting the meeting that **finishes earliest**, the algorithm continuously creates the largest possible remaining time interval for future meetings.

The main advantage of this approach is its efficiency. Instead of checking every possible combination of meetings, which could require exponential time, the greedy algorithm sorts the activities once and then scans them sequentially. Thus, the overall complexity is **O(n log n)**.

20. Conclusion

The **Activity Selection Problem** is an important example of a **Greedy Algorithm** used for scheduling optimization. In the software company scenario, each client meeting is represented by its start and finish times. By sorting meetings according to their finishing times and repeatedly selecting the earliest-finishing compatible meeting, the company can schedule the **maximum number of non-overlapping meetings**.

The greedy approach is optimal because selecting the earliest finishing activity leaves the maximum possible time for the remaining activities. With an overall time complexity of **O(n log n)**, it provides a fast and practical solution for large scheduling problems. Therefore, the technique is highly suitable for **client meeting scheduling, resource allocation, interviews, conferences, and other time-based scheduling systems**.